1. Spécificités du jeu de carte :

- Créez un jeu standard de 52 cartes.
- Entrez les noms des joueurs.
- Limitez le nombre de joueurs à cinq
- Mélangez le jeu de cartes
- Distribuez une carte à chaque joueur (face cachée)
- Retournez les cartes des joueur, en montrant quelle carte ils ont
- Vérifiez quel joueur a la carte la mieux classée:
    - As > Roi > Reine > Valet > 10 >... 2
- En cas d'égalité, le gagnant est basé sur la couleur:
    - Trèfles > piques > cœurs > carreaux Afficher le nom et la carte du joueur gagnant
- Remettez toutes les cartes dans le paquet
- Revenez à l'étape de mélange

2. Comment planifier le modèle ?

Identifions d'abord les principaux éléments que les joueurs verront et avec lesquels ils interagiront.
Le moyen le plus simple pour déterminer les éléments du modèle est de lire les exigences et d'y repérer les noms importants.

Dans la description, on trouve :

- un jeu de cartes (une collection de cartes à jouer),
- des joueurs,
- un rang de carte,
- une couleur de carte.

Reste à établir comment relier les joueurs à leurs cartes.
Un joueur possède une carte pendant la partie, n'est-ce pas ?
Même si ce n'est pas formulé explicitement, il est logique de regrouper les cartes d'un joueur dans une classe Hand.
Cela respecte le principe de responsabilité unique.

Un joueur est donc constitué de :

- son nom,
- sa main (Hand).

Ainsi, si les règles changent, cela affectera probablement :

- le nombre de cartes dans une main,
- ou la manière dont les cartes sont manipulées.

En revanche, le modèle d'un joueur (son identité) ne devrait pas avoir à changer lorsque vous jouez à un jeu différent.

2.1.Générer le rang et la couleur

Deux classes spécifiques de types "enum" devront être créées.
Attribuer des nombres entier aux valeurs de rang et couleur afin de pouvoir les comparer par la suite pour déterminer le gagnant.

2.2.Générer les cartes à jouer

Cette classe peut désormais utiliser les éléments définis à l'étape 1.
Le rang et la couleur d'une carte ne changent pas au cours de la partie : ce sont des attributs immuables.

En revanche, selon l'énoncé, nous devons pouvoir retourner une carte. Nous devons donc ajouter un attribut indiquant si la carte est face visible ou face cachée.
Contrairement au rang et à la couleur, cet attribut pourra varier pendant la partie.

2.3.Générer le tas/pile

Il s'agit d'une collection de cartes à jouer dont l'ordre importe : une ArrayList convient parfaitement.
L'utilisation de boucles permettra de remplir cette collection à partir des rangs et couleurs définis précédemment.

Une fois le tas créé, il doit contenir des cartes en désordre.
Il faut donc implémenter une méthode permettant de mélanger la pile.
On rappelle qu'il est possible d'utiliser la méthode Collections.swap pour réaliser un mélange simple.

Le tas doit ensuite permettre :

- de retirer la première carte,
- d'ajouter une carte à nouveau dans la pile.

2.4.Générer la main et les joueurs

Comme pour le tas, une main est une collection de cartes à jouer.
Elle est vide au départ, mais doit pouvoir être remplie au cours du jeu.
On doit également être en mesure :

- de voir le contenu de la main,
- d'ajouter une carte,
- d'en retirer une.

Pour les joueurs, un nom est indispensable.
Un joueur doit pouvoir :

- présenter sa main,
- y ajouter une carte,
- et en retirer une.

3. Comment planifiez le contrôleur?

Le contrôleur est responsable du séquençage des interactions entre l'utilisateur et le jeu.
En observant le déroulement d'une partie, la séquence est la suivante :

1. Créer le deck
2. Saisir les noms des joueurs
3. Mélanger les cartes
4. Distribuer une carte à chaque joueur
5. Retourner les cartes
6. Évaluer le gagnant
7. Afficher le gagnant
8. Reconstituer le deck

Le contrôleur doit donc connaître l'étape courante du jeu et déterminer quelles entrées sont valides à cette étape.

COMMENT LE CONTRÔLEUR SAIT CE QUI LUI A ÉTÉ INTRODUIT ?

La commande d'entrée provient de la vue.
Le contrôleur reçoit ces entrées et doit décider comment réagir.

La prochaine étape consiste à analyser la séquence ci-dessus pour identifier clairement :

- quelles actions doivent être exécutées par le contrôleur,
- quelles requêtes doivent être adressées à la vue,
- et quelles informations le contrôleur doit lire ou valider.

```
import java.util.ArrayList;

import cardgame.model.Deck;

public class View {
  GameController gc;

  public void promptSomething(){}

  public void showSomething(){}

  public void setController(GameController gc) {
    this.gc =gc;
  }
};


public class GameController {

  private enum GameState{
    AddingPlayer,
    CardsDealt,
    WinnerRevealed
  };

  GameState gameState;
  Deck deck;
  View view;

  public GameController(Deck deck) {
    this.deck = deck;
    this.view = new View();
    view.setController(this);
    gameState = GameState.AddingPlayer;
  }
}
```

3.1.Démarrer le jeu

La première étape consiste à instancier le contrôleur dans le main.
Le contrôleur construit les objets de base du jeu :

- le tas (deck),
- une liste vide de joueurs,
- et il doit connaître la vue.

Pour respecter correctement le modèle MVC, la vue doit être placée dans son propre package, instanciée ailleurs, puis transmise au contrôleur plutôt que créée directement par celui-ci.

En effet, si l'application devait un jour disposer d'une version mobile, il faudrait lui fournir une autre interface utilisateur.
Si le contrôleur créait lui-même sa vue, il faudrait alors modifier son code, ce qui violerait le principe d'inversion de dépendance (D de SOLID).

Le contrôleur ne doit dépendre que d'une abstraction de la vue, afin qu'il soit possible de changer le type de vue sans changer le contrôleur.

3.2.Saisie des noms des joueurs

Pour chaque nom saisi, le contrôleur ajoute un joueur à la liste via la méthode addPlayer.
Lorsque tous les noms ont été saisis, la vue indique que l'entrée est terminée.

La méthode startGame met alors fin à la phase d'ajout :
une fois la partie lancée, aucun joueur supplémentaire ne peut être ajouté.

3.3.Donner les cartes

1. Distribution
   - Le contrôleur mélange le paquet.
   - Il retire la première carte et la place dans la main de chaque joueur.
   - Il demande ensuite à la vue d'afficher l'état du jeu : chaque joueur a désormais une carte face cachée.

2. Révélation et évaluation
   - Le contrôleur retourne les cartes de tous les joueurs via flipCards.
   - Le gagnant est déterminé grâce à evaluateWinner.
   - La vue affiche alors le résultat avec displayWinner.
   - Enfin, les cartes sont récupérées et remises dans le paquet.

3. Rejouer
   - Le contrôleur reconstruit le paquet via rebuildDeck.
   - L'état du jeu revient à l'étape 3 pour permettre une nouvelle manche.

3.4.Voici maintenant l'organisation de votre contrôleur.

```
public class GameController {
    public GameController(View view, Deck deck) {}

    public void addPlayer(String playerName) {}
    public void startGame() {}
    public void flipCards() {}

    void evaluateWinner() {}
    void displayWinner() {}
    void rebuildDeck() {}
};
```

3.5.Comment planifiez-vous la vue?

La vue sert à la fois d'interface utilisateur et de source d'événements.

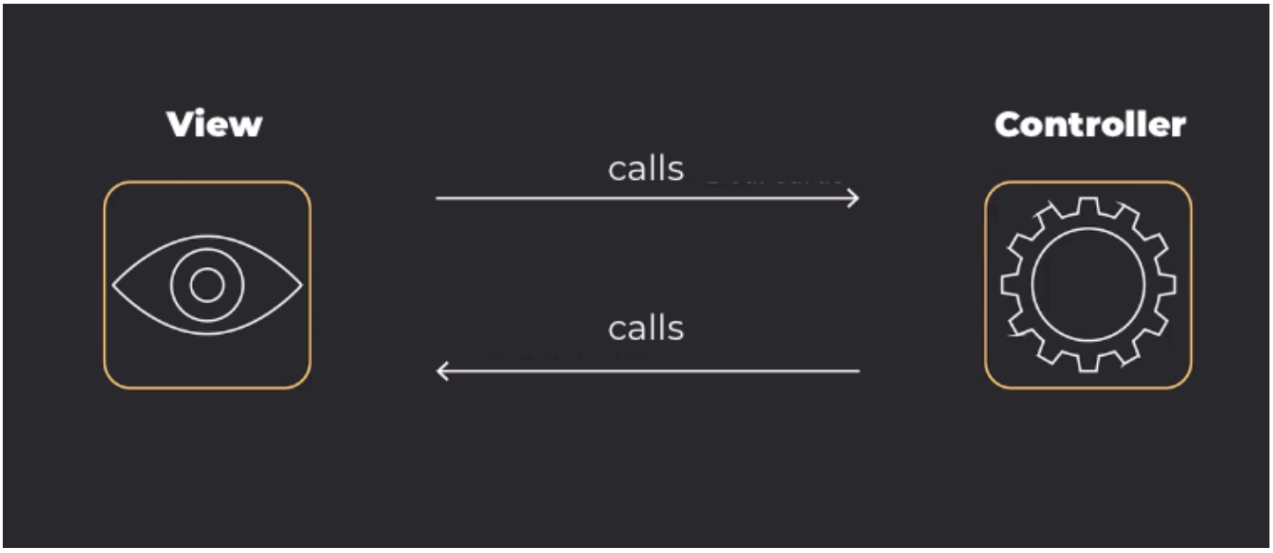
Le contrôleur ne fait qu'une chose avec elle : lui demander d'afficher des informations.

Pour rester cohérent avec la structure de l'application, la vue ne manipule jamais directement les objets du modèle. Elle reçoit uniquement ce dont elle a besoin pour afficher :

- le nom d'un joueur,
- ou les informations nécessaires d'une carte (rang, couleur).

Ainsi, le modèle reste responsable de l'état interne, et la vue reste responsable de l'affichage.

3.6.Interactions entre la vue, le contrôleur et l'état du jeu



Le changement d'étape du jeu suit le principe du pattern State présenté en cours : le comportement du contrôleur dépend de l'état courant du jeu.

Le contrôleur réagit aux entrées de la vue en fonction de l'état courant du jeu.
L'état ("AddingPlayer", "CardsDealt", "WinnerRevealed", etc.) détermine quelles actions sont possibles et ce que la vue doit afficher.

3.6.1.Saisie des noms des joueurs (state : AddingPlayer)

addPlayer()

- La vue récupère chaque nom saisi et l'envoie au contrôleur.
- Le contrôleur ajoute le joueur dans sa liste.
- La vue affiche alors le nom ajouté : showPlayerName(playerIndex, name).

startGame()

- le contrôleur mélange le jeu (
- le contrôleur distribue une carte à chaque joueur.
- La vue affiche les cartes face cachée pour chaque joueur : showFaceDownCardForPlayer(playerIndex).

Quand tous les noms sont saisis, le contrôleur change d'état pour passer à la phase suivante.

3.6.2. Cartes distribuées (state : CardsDealt)

flipCards()

- Le contrôleur retourne toutes les cartes

- Le contrôleur montre les cartes.
- Détermine le gagnant.
- La vue affiche :
- showCardForPlayer(playerIndex, rank, suit)
- showWinner(winnerName).

Après la révélation du gagnant, le contrôleur passe à l'état suivant.

3.6.3.Gagnant révélé (state : WinnerRevealed)

Lorsque l'affichage est terminé, le contrôleur peut réinitialiser le jeu ou revenir à un état précédent.

3.7.Comment implémentez-vous la vue?

```
1 public class View {
2     public void showPlayerName(int playerIndex, String name);
3     public void showFaceDownCardForPlayer(int playerIndex, String name);
4     public void showCardForPlayer(int playerIndex, String Name, String rank, String suit);
5     public void showWinner (String winnerName);
6 };
```

Notre mise en œuvre est simple. Il montre simplement les noms des joueurs, les cartes distribuées et les résultats sur la ligne de commande. Mais, puisque nous avons séparé cette fonctionnalité, nous pourrions facilement remplacer le mécanisme par une implémentation GUI.

4. Appliquons les principes SOLID à notre code

4.1.Point de départ : une seule règle d'évaluation du gagnant

Au début, l'évaluation du gagnant est directement écrite dans le contrôleur.
Il n'existe qu'une seule règle : la carte la plus haute gagne.
Le contrôleur s'occupe donc de deux choses :

- le déroulement du jeu,
- la logique d'évaluation.

Cela mélange des responsabilités qui devraient être séparées.
Si en plus on souhaite y introduire une nouvelle façon d'évaluer…

```
1 evaluateWinner(List<Player> players, bool findHighCardWinner) {
2    if (findHighCardWinner) { /* find them */ }
3    else { /* find the other */ }
4 };
```

4.2.Application du principe SRP (Responsabilité Unique)

Pour clarifier le rôle du contrôleur, on extrait l'évaluation du gagnant dans une classe dédiée.
Le contrôleur ne décide plus du vainqueur : il demande simplement le résultat.
Cette séparation rend le code plus clair, mais fonctionne seulement tant qu'il n'existe qu'une seule règle.

java

```
class GameEvaluator {
    Player evaluateWinner(List<Player> players) { ... }
}
```

4.3.Apparition du problème O/C (Open/Closed)

Dès qu'on souhaite introduire une deuxième règle (par exemple, la carte la plus basse gagne), le système commence à se dégrader.

Ajouter un paramètre ou des conditions supplémentaires rendrait la logique :

- plus difficile à lire,
- plus difficile à tester,
- et violerait le principe Open/Closed :

à chaque nouvelle règle, il faudrait modifier du code existant.

Le système n’est plus extensible.

4.4.Transformation vers une interface : naissance du Strategy

Pour éviter ces modifications successives, l’évaluation devient une interface représentant une règle du jeu.
Chaque règle est alors une implémentation distincte.
Le contrôleur n’a plus besoin de savoir quelle règle est utilisée :
il utilise toujours la même interface.

```
1 public interface GameEvaluator {
2     public Player evaluateWinner(List<Player> players);
3 }
4
5 public class HighCardGameEvaluator implements GameEvalutor {
6 }
7 public class LowCardGameEvaluator implements GameEvalutor {
8 }
```

On obtient une structure flexible permettant d’ajouter librement de nouvelles règles sans modifier le contrôleur.

4.5.Injection de dépendance : comment utiliser la règle choisie

Le contrôleur ne doit pas créer lui-même la règle du jeu.
La règle lui est fournie depuis l’extérieur, au moment de sa création.

C’est l’injection de dépendance :
l’objet dont le contrôleur dépend est transmis plutôt que instancié à l’intérieur.

Ainsi :

- le contrôleur reste inchangé,
- la logique de la règle peut évoluer librement,
- et toute nouvelle règle est simplement ajoutée comme une nouvelle implémentation.

```
1 public GameController (GameEvaluator _gameEvaluator) {
2    gameEvaluator = _gameEvaluator;
3 }
```

4.6.Création de l’interface GameViewable

```java
public interface GameViewable {
    void setController(GameController controller);

    void promptForPlayerName();
    void showPlayerName(int playerIndex, String name);

    void showFaceDownCardForPlayer(int playerIndex, String name);

    void promptForFlip();
    void showCardForPlayer(int playerIndex, String name, String rank, String suit);
    void showWinner (String winnerName);

    void promptForNewGame();
}

```

4.6.1.pourquoi et quoi respecter ?

Nous créons l’interface GameViewable pour pouvoir remplacer facilement la vue
(par exemple, passer de la console à une interface graphique) sans modifier le contrôleur.
Le contrôleur travaille uniquement avec cette interface, ce qui permet de changer le type de vue librement.

4.6.2.Et GameViewable, c’est quel principe ou pattern ?

GameViewable illustre principalement :

- Le principe d’Interface Segregation (I de SOLID)
  → une interface petite, ciblée, cohérente.
- Le principe Open/Closed (O de SOLID)
  → on peut ajouter de nouvelles vues sans toucher au contrôleur.
- Le découplage MVC
  → la vue et le contrôleur communiquent via une interface, pas par des classes concrètes.

Ce n’est pas un design pattern à lui tout seul, mais c’est une construction utilisée dans plusieurs patterns structurels
(Strategy, Adapter, Facade).
Ci dessous un contre exemple sur une interface possédant des méthodes non conformes.

```java
public interface GameViewable {
    // bad placement of this idea
    public void restartGame() {
        // doing things without being told
        clearOutCardDisplay();
        resetNames();
    }
};
```

4.7.Inversion de dépendance et ajout d'une nouvelle vue

Avant d'ajouter une nouvelle vue (par exemple une interface graphique), il faut vérifier que la vue dépend du contrôleur, et non l'inverse.
Le contrôleur doit uniquement connaître l'interface GameViewable, jamais une vue concrète.

Ainsi, toute nouvelle vue doit simplement implémenter GameViewable et se brancher dans le contrôleur sans modifier celui-ci.

L'objectif est de s'assurer que :

- le contrôleur utilise uniquement GameViewable,
- la vue reçoit son contrôleur via setController(...),
- aucun appel direct n'est fait vers une vue spécifique dans le contrôleur,
- et aucune logique métier n'est ajoutée dans la vue.

Si ces conditions sont remplies, il devient possible d'ajouter une vue Swing (comme GameSwing) en l'implémentant simplement.
Le contrôleur n'est jamais modifié : il fonctionne avec n'importe quelle vue respectant l'interface.

```
1 public void addPlayerName(String playerName) {
2    // this knows too much about the view's implementation
3    view.textArea.append("[" + playerIndex + "][" + playerName +"]\n");
4 }
```

4.7.1.Phrase de transition

« Si votre architecture respecte déjà l'inversion de dépendance (le contrôleur dépend de l'interface GameViewable et non d'une vue concrète), alors vous pouvez brancher une nouvelle vue comme GameSwing sans modifier le contrôleur. »

1. Le contrôleur définit l'interface de la vue → GameViewable.
2. La vue (CommandLineView, GameSwing, etc.) implémente cette interface.
3. Le contrôleur ne connaît que GameViewable, pas la vue graphique.
4. La vue reçoit son contrôleur via setController().
5. Une nouvelle vue peut être ajoutée sans changer le contrôleur.

4.7.2.Phrase finale TD pour guider le câblage Swing

« Une fois votre code découplé, vous pouvez simplement instancier GameSwing à la place de CommandLineView, l'attacher au GameController via setController(), puis appeler createAndShowGUI() pour afficher l'interface. Aucune modification du contrôleur n'est nécessaire.»

```java
package com.r304.cardgame.view;

import java.awt.Component;
import java.awt.Container;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JComponent;
import javax.swing.JFrame;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;

import com.r304.cardgame.controller.GameController;

public class GameSwing implements GameViewable {
    GameController controller;

    JButton btnAddPlayer;
    JButton btnDealCards;
    JButton btnFindWinner;
    JTextArea textArea;
    char nextPlayerName = 'A';

    public void createAndShowGUI() {
                // create the main display area, with enough initial space
        JFrame frame = new JFrame("MVC-SOLID-Game");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(500, 500);

        // all of the controls will appear vertically
        Container contentPane = frame.getContentPane();
        contentPane.setLayout(new BoxLayout(contentPane, BoxLayout.Y_AXIS));

        addAddPlayerButton(contentPane);
        addDealCardsButton(contentPane);
        addFindWinnerButton(contentPane);

        addControllerCommandTracker(contentPane);

        frame.setVisible(true);
    }

    // when clicked, tell controller to add a player of the given name
    // this SHOULD bring up a dialog box to allow the user to enter a name
    // but that is beyond the scope of what we need to learn
    // for simplicity, we'll just tell it to add "Player A", "Player B", etc.
    private void addAddPlayerButton (Container contentPane) {
        btnAddPlayer = new JButton("Add A Player");
        addCenteredComponent (btnAddPlayer, contentPane);
        btnAddPlayer.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                textArea.append("Current players:\n");
                controller.addPlayer("Player " + nextPlayerName++);
            }
        });
    }

    // when clicked, tell controller to deal cards to players
    private void addDealCardsButton (Container contentPane) {
        btnDealCards = new JButton("Deal Cards");
        addCenteredComponent(btnDealCards, contentPane);
        btnDealCards.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                textArea.append("Cards dealt:\n");
                controller.startGame();
            }
        });
    }

    // when clicked, tell controller to flip cards and determine who won
    private void addFindWinnerButton(Container contentPane) {
                        btnFindWinner = new JButton("Find Winner");
                        addCenteredComponent(btnFindWinner, contentPane);
                        btnFindWinner.addActionListener(new ActionListener() {
                            public void actionPerformed(ActionEvent e) {
                                textArea.append("Show your cards:\n");
                                controller.flipCards();
                            }
                        });
    }

    // a simple place to display what the controller is telling us
    // very similar to our command line version
    private void addControllerCommandTracker(Container contentPane) {
                        textArea = new JTextArea("Game Status\n", 100, 1);
                        JScrollPane scrollPane = new JScrollPane(textArea);
                        addCenteredComponent(scrollPane, contentPane);
                        textArea.setSize(500, 500);
    }

                    // make sure that every time something is added to the text area,
                    // scroll down to the bottom so that it is visible
                    // textArea control does not have an auto-scroll option
                    // so we have to do it ourselves
                    private void appendText (String text) {
                                        textArea.append(text + "\n");
                                        textArea.setCaretPosition(textArea.getDocument().getLength());
                    }

    // all controls are added so they are centered horizontally in the area
    private void addCenteredComponent(JComponent component, Container contentPane) {
        component.setAlignmentX(Component.CENTER_ALIGNMENT);
        contentPane.add(component);
    }

    // GameViewable interface implementations
                    public GameSwing() {
                                        // TODO Auto-generated constructor stub

                    }

                    @Override
                    public void setController(GameController gc) {
                                        this.controller = gc;

                    }

                    // same implementation as our CommandLineView
                    // just show what the controller is telling us
    @Override
    public void showPlayerName(int playerIndex, String playerName) {
                    appendText("[" + playerIndex + "][" + playerName +"]");
    }

    @Override
    public void showCardForPlayer(int playerIndex, String playerName, String cardRank, String cardSuit) {
                    appendText("[" + playerName + "][" + cardRank + ":" + cardSuit + "]");
    }

    @Override
    public void showWinner(StringBuilder winnerName) {
        String wName = winnerName.toString();
                    appendText("Winner!\n" + wName);
    }

                    @Override
                    public void showFaceDownCardForPlayer(int playerIndex, String name) {
                                        appendText("[" + name + "][][]");
                    }


                    @Override
                    public void promptForPlayerName() {
                                        System.out.println("Enter Player Name:");
                                        // it's ok to add names
                        btnAddPlayer.setEnabled(true);

                        // it's ok to start, one a player has been added
                        btnDealCards.setEnabled(nextPlayerName > 'A');

                        // but not to ask to find a winner
                        btnFindWinner.setEnabled(false);
                    }

                    @Override
                    public void promptForFlip() {
                                        System.out.println("Press enter to reveal cards");

                                        // past the time to add names
                                        btnAddPlayer.setEnabled(false);

                                        // or deal cards
                                        btnDealCards.setEnabled(false);

                                        // but you can ask to find a winner
                                        btnFindWinner.setEnabled(true);
                    }

                    @Override
                    public void promptForNewGame() {
                                        System.out.println("Press enter to deal again");

                                        // past the time to add names
                                        btnAddPlayer.setEnabled(false);

                                        // ok to deal cards for the next game
                                        btnDealCards.setEnabled(true);

                                        // can't find a winner yet
                                        btnFindWinner.setEnabled(false);
                    }

                    @Override
                    public void showTieBreak() {
                                        // TODO Auto-generated method stub

                    }
}
```